1. Objective

The goal is to classify 28×28 grayscale images of handwritten digits (0–9) for the Kaggle Digit Recognizer competition, using PyTorch. The metric is accuracy on 28,000 unlabelled test images.

I set three concrete targets:

1. Build an MLP baseline and confirm it reaches the reference accuracy of about 0.97.
2. Design a CNN with at least two convolutional blocks and reach 0.99 or more.
3. Improve the CNN with at least one technique and measure its effect against the plain CNN.

All reusable code (data loading, models, training, prediction, plots) lives in my own package, `digits`, and the notebook imports from it.

Final result: **[0.99471] public score on Kaggle** (username: **[manvig1]**).

2. Data preparation

`train.csv` has 42,000 labelled images, each stored as one row of 784 pixel values (0–255) plus a label. `test.csv` has 28,000 unlabelled images. All preparation is in `src/digits/data.py`.

Scaling and standardisation. I divide pixels by 255 to get values in 0–1, then subtract the mean (0.131) and divide by the standard deviation (0.309). Zero-centred inputs on a similar scale give more stable gradients and faster convergence. The mean and std are computed on the **training split only** and reused for validation and test, so no information from those sets leaks into training.

Reshaping. Each 784-value row is reshaped to [1, 28, 28], so batches have shape [B, 1, 28, 28] (batch, channels, height, width). The MLP flattens this again, but the CNN needs the 2D layout to know which pixels are neighbours.

Validation split. I hold out 10% (4,200 images) with a stratified split, so every digit keeps the same proportion in train and validation. I checked this: validation counts range from 380 (digit 5) to 468 (digit 1), mirroring the full dataset. The split, shuffling and weight initialisation use a fixed seed (42) so results are reproducible.

Data loaders. Training batches of 128 are shuffled every epoch so the model cannot learn the order of examples. Validation and test loaders are not shuffled. For the test set this is essential, because row *i* must stay matched to ImageId *i*.

Data augmentation (training only). For the improved model, each training image is randomly transformed every time it is loaded:

- rotation up to ±10°, because handwriting is often slanted;

- shift up to 10% (about 3 pixels), because digits are not always centred;
- zoom between 90% and 110%, because people write larger or smaller.

I deliberately used **no flips**: a mirrored 2 or 7 is not a valid digit, and a flipped 6 can become a 9. Empty corners created by rotation are filled with the normalised value of a black pixel (about −0.42), not 0, so no artificial grey patches appear. The validation set is never augmented, so it measures performance on real digits like those in the test set.

Sanity checks. Before training I verified the batch shape ([128, 1, 28, 28]), plotted 16 images with their labels to confirm they match, and checked the class balance of the split.

3. Models and architectures

All models are in `src/digits/models.py` and output 10 raw scores (logits), one per digit. There is no softmax layer, because `CrossEntropyLoss` applies it internally; adding it again would distort training.

Model	Architecture	Parameters
MLP (baseline)	Flatten → Linear 784→256 → ReLU → Linear 256→128 → ReLU → Linear 128→10	~235k
CNN	2 conv blocks (32, then 64 filters) → Flatten → Linear 3136→128 → ReLU → Linear 128→10	~468k
Improved CNN	Same CNN + BatchNorm after every convolution + Dropout (p = 0.4) in the classifier	~468k

MLP baseline. The MLP flattens each image into 784 independent inputs. It works reasonably well but ignores the 2D structure: it does not know that neighbouring pixels are related, and must learn the same stroke separately at every position. It gives a reference point for how much the CNN actually adds.

CNN. Each convolutional block is Conv 3×3 → ReLU → Conv 3×3 → ReLU → MaxPool 2×2.

- *3×3 kernels*: two stacked 3×3 convolutions cover a 5×5 area with fewer parameters than one 5×5 filter, plus an extra non-linearity.
- *Padding of 1*: keeps the image size unchanged through each convolution, so edge pixels are not lost.

- *Max pooling*: halves height and width (28 → 14 → 7), which reduces computation and makes the model tolerant to small shifts.
- *Channels 1 → 32 → 64*: as spatial resolution shrinks, more filters are needed to describe more complex features.

Convolutions use **weight sharing**: the same small filter slides across the whole image, so a feature such as a curved stroke is learned once and detected anywhere. The first block learns edges and stroke fragments; the second combines them into loops, corners and line endings. Most parameters (~401k) are in the first dense layer, while the four convolutional layers have only ~65k, so the CNN's advantage comes from its structure, not its size.

Improvement. The plain CNN overfitted (see Section 5), so I chose techniques that fight overfitting:

- **BatchNorm** re-standardises activations after each convolution, which stabilises and speeds up training and allows a higher learning rate.
- **Dropout (0.4)** randomly zeroes 40% of the classifier's inputs during training, so the network cannot rely on any single neuron. It sits in the classifier because that is where most parameters are.
- **Data augmentation** (Section 2) shows the model a different variation of each digit every epoch, so it cannot memorise exact pixel positions.
- **OneCycleLR** scheduler (Section 4).

BatchNorm and Dropout are implemented as options on the same `CNN` class, so the plain and improved models differ only in those settings. This makes the comparison controlled.

4. Training and validation

Training code is in `src/digits/train.py`. Each batch goes through the standard loop: clear old gradients, forward pass, compute the loss, backpropagate, update the weights.

Setting	MLP / CNN / CNN+BN+Dropout	Improved CNN (final)
Loss	Cross-entropy	Cross-entropy
Optimiser	Adam	Adam
Learning rate	0.001, constant	OneCycleLR, peak 0.003

Epochs	10	15
Batch size	128	128
Augmentation	No	Yes
Device	Apple MPS GPU	Apple MPS GPU

Why these choices. Cross-entropy is the standard loss for multi-class classification; it penalises confident wrong answers heavily. Adam adapts the step size for each weight and works well with its default learning rate of 0.001, which makes it a reliable choice for a fair comparison.

OneCycleLR. The learning rate warms up to a peak and then decreases to almost zero, updated after every batch. The high phase moves the weights quickly toward a good region; the low phase lets them settle precisely. A higher peak (0.003) is safe because BatchNorm stabilises training. The improved model trains for 15 epochs because augmentation makes the task harder and it no longer overfits.

Validation. After every epoch, each model is evaluated on the 4,200 held-out images in evaluation mode (`model.eval()` and no gradients), so Dropout is off and BatchNorm uses its stored statistics. I record training and validation loss and accuracy per epoch to plot learning curves. Whenever validation accuracy reaches a new best, I save the weights, and restore the best ones at the end. This acts as a simple form of early stopping.

Controlled comparison. I changed one thing at a time: MLP → CNN (architecture), → BatchNorm + Dropout (regularisation), → augmentation + OneCycleLR (training). The same seed and split are used throughout, so each difference can be attributed to a specific change.

5. Results and Kaggle score

The improved CNN reached **0.9952** validation accuracy and 0.99471 on the Kaggle public leaderboard, cutting errors by about 78% compared with the MLP baseline.

Model	Best val accuracy	Errors (of 4,200)	Error rate

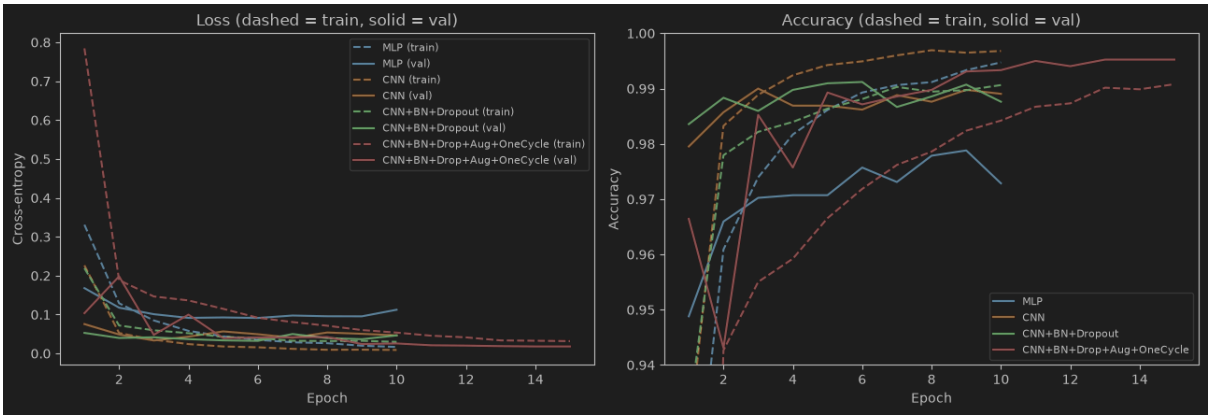
MLP (baseline)	[0.97xx]	~92	~2.2%
CNN	0.9900	42	1.0%
CNN + BN + Dropout + Augmentation + OneCycleLR	**0.9952**	**20**	**0.48%**

Near 99%, accuracy differences look small, so error counts are more informative: each step roughly halved the number of mistakes.

Kaggle. Username: manvig1. Public score: 0.99471. The submitted file (`results/submission.csv`) has 28,000 rows, the header `ImageId,Label` and integer labels 0–9, all checked in code before upload. The predicted class distribution (from 2,505 fives to 3,193 ones) mirrors the training distribution.

Learning curves (`results/training_curves.png`):

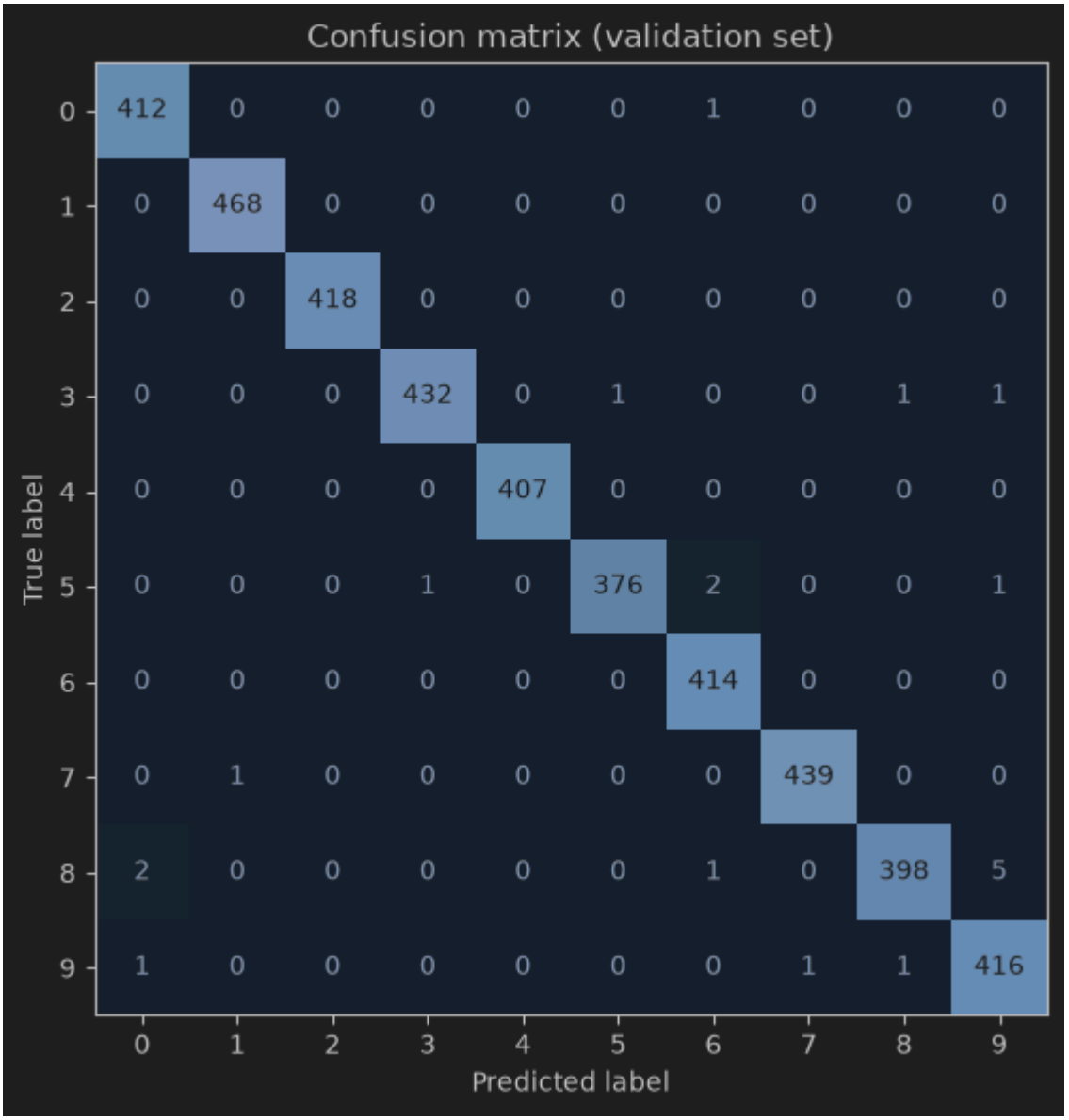
- *MLP*: validation loss is lowest around epoch 4–6 and then rises while training loss keeps falling. It reaches its limit early.
- *CNN*: validation loss is lowest at epoch 3 (0.033) and rises to 0.047 by epoch 10, while training accuracy (0.997) ends well above validation (0.989). This is overfitting: the model starts memorising training images.
- *CNN + BN + Dropout*: a smaller train–validation gap, but validation accuracy still plateaus around 0.99.
- *Improved CNN*: training accuracy stays **below** validation for the whole run, and validation loss keeps falling to 0.018 at epoch 15. Overfitting is gone. Validation accuracy dips at epochs 2 and 4 during OneCycle's high learning-rate phase, then the largest gains come in epochs 9–15 as the rate decreases.



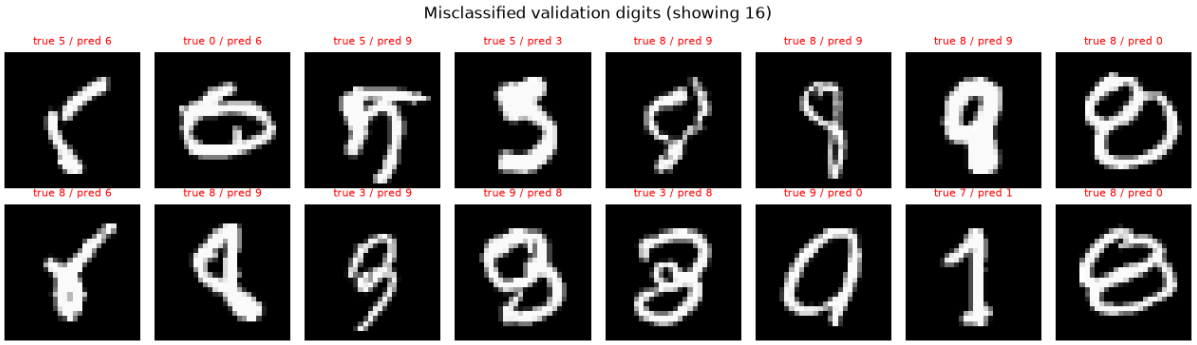
Confusion matrix (`results/confusion_matrix.png`). The 20 validation errors of the improved CNN are concentrated in a few digits:

True digit	Errors	Predicted as
8	8	9 (×5), 0 (×2), 6
5	4	6 (×2), 3, 9
3	3	5, 8, 9
9	3	0, 7, 8
0	1	6
7	1	1
1, 2, 4, 6	0	—

Digit 8 is the hardest (98.0% correct), and 8 → 9 is the most frequent confusion: an 8 with an open lower loop looks like a 9, while the reverse rarely happens. Almost all errors involve round digits (8, 5, 3, 9, 0, 6), which differ only in small details such as whether a loop is closed.



Misclassified digits. Most misclassified digits are badly written or ambiguous; in about X of 16 cases, I could not confidently tell the true label myself.



6. Conclusions and limitations

Conclusions

1. **Structure matters more than size.** The CNN has about twice the parameters of the MLP, but most of them are in its dense layer. It wins because convolutions use the spatial structure of images: local patterns and weight sharing. It beat the MLP's final accuracy after a single epoch.
2. **The main problem was overfitting, not capacity.** The plain CNN learned the training set almost perfectly (0.997) but validation loss started rising after epoch 3. Adding more layers would not have helped.
3. **Regularising the training data helped most.** BatchNorm and Dropout gave a modest gain on their own. Adding data augmentation and OneCycleLR removed the overfitting and halved the errors again, from 1.0% to 0.48%.
4. **Error counts are a better lens than accuracy near 99%.** Going from 0.990 to 0.995 looks small, but it is half as many mistakes.
5. **Remaining errors are concentrated and often ambiguous.** Most involve round digits, especially 8 read as 9, and several misclassified images are hard even for a person to read.

Limitations

- *Single validation split.* All comparisons rely on one 90/10 split, so small differences (for example, 0.990 vs 0.991) may be within random variation. Cross-validation or several seeds would give more reliable estimates.
- *Slightly optimistic validation score.* I selected the best epoch using the validation set, so the reported validation accuracy is a little optimistic. The Kaggle score is the independent estimate.
- *Limited tuning.* Learning rate, dropout rate, augmentation strength and architecture were set to sensible defaults, not systematically tuned.
- *Reproducibility on the Apple GPU.* Some MPS operations are not fully deterministic, so repeated runs can differ slightly in the last decimals even with a fixed seed.
- *Ambiguous labels.* Some digits are genuinely ambiguous, which puts a ceiling on achievable accuracy.

Possible improvements: an ensemble of several CNNs trained with different seeds (averaging their logits), retraining on all 42,000 images, a deeper network with a third convolutional block, and k-fold cross-validation for more reliable comparisons.